How to Serve Your Sandwich? MEV Attacks in Private L2 Mempools

Krzysztof M. Gogol[1,2], Manvir Schneider[3], Jan Gorzny[2], and Claudio J. Tessone[1,4]

[1] University of Zurich UZH
[2] Zircuit
[3] Cardano Foundation
[4] UZH Blockchain Center

Abstract. We study the feasibility, profitability, and prevalence of sandwich attacks on Ethereum rollups with private mempools. First, we extend a formal model of optimal front- and back-run sizing, relating attack profitability to victim trade volume, liquidity depth, and slippage bounds. We complement it with an execution-feasibility model that quantifies co-inclusion constraints under private mempools. Second, we examine execution constraints in the absence of builder markets: without guaranteed atomic inclusion, attackers must rely on sequencer ordering, redundant submissions, and priority fee placement, which renders sandwiching probabilistic rather than deterministic. Third, using transaction-level data from major rollups, we show that naive heuristics overstate sandwich activity. We find that the majority of flagged patterns are false positives and that the median net return for these attacks is negative. Our results suggest that sandwiching, while endemic and profitable on Ethereum L1, is rare, unprofitable, and largely absent in rollups with private mempools. These findings challenge prevailing assumptions, refine measurement of MEV in L2s, and inform the design of sequencing policies.

Keywords: MEV · Sandwich Attacks · Rollups · Private Mempools

1 Introduction

In March 2025, in block `22029771`, a trader attempted to swap 220 000 USDC into USDT on Uniswap v3 on Ethereum. The outcome was unexpected: the trader received only about 5000 USDT. This unfavorable result was due to a sandwich attack.

Sandwich attacks are among the most detrimental forms of Maximal Extractable Value (MEV) [3], a form of influencing transaction ordering within a blockchain block. In such an attack, an adversary—typically an automated bot—detects a large pending transaction in the mempool and strategically places one transaction immediately before and another immediately after the victim's. By exploiting predictable price movements, the attacker captures profit while the

victim suffers from significantly worse execution. Sandwiching illustrates the harmful side of MEV, though not all MEV is necessarily detrimental [5].

Rollups [8,10] are popular Ethereum Layer-2 (L2) solutions that primarily aim to improve scalability. However, they may also deploy mechanisms to mitigate (harmful) MEV [1]. A common countermeasure against sandwich attacks is the removal of the public mempool in favor of a private pool managed by a centralized sequencer [2]. This approach, adopted by several major L2, prevents adversaries from observing transactions in advance. Nevertheless, early empirical evidence indicates that sandwiching patterns also occur on these L2s, despite private mempools [12].

On rollups with fast pre-confirmations (e.g., inclusion receipts), searchers cannot guarantee atomic bundling of front- and back-run transactions, as most sequencers operate private mempools without open builder markets. Instead, bots maximize the probability of same-block inclusion via (i) *timing* (predicting proposer cadence and batching windows), (ii) *fee placement* (base fee + priority tips calibrated to sequencer policy), and (iii) *redundancy* (parallelized, nonce-staggered submissions with protective slippage bounds). The absence of public bundle markets shifts MEV from strictly atomic execution to *probabilistic* execution, which we capture in the reward design (Section 2).

Related Work. Qin et al. [7] provide one of the earliest systematic measurements of MEV on Ethereum, including sandwich attacks. It identifies attacker trades by examining in-block swap transactions and linking them either through sender addresses (for EOAs) or recipient addresses (for contracts). Our empirical approach follows this identification strategy and adapts it to L2-specific AMM dynamics (such as cyclic arbitrage trades) and shared routers (e.g. Uniswap v4). Complementary research [14,13] models sandwich profitability under Constant Product AMMs (CPMM) such as Uniswap v2. In contrast, we develop an attacker-profitability model for both CPMMs and concentrated-liquidity AMMs (CLMMs) such as Uniswap v3 and derive an optimal attacker strategy.

The study of MEV on rollups remains at an early stage. Torres et al. [12] provide the first estimations on L2s, with private-mempool. They identified cyclic arbitrage and searched for sandwich attack patterns. Their analysis is based on the ERC20 token transfers matching and matching attacker transactions by corresponding sender and receiver. Due to the mechanics of MEV bot smart contracts, as well as router mechanism introduced by Uniswap (v4), we follow the methodology of Qin et al. [14,13]. for matching attacker transaction and base it on swap events, rather than ERC20 transfers.

Further analysis of MEV on L2 include atomic and non-atomic arbitrage. Solmaz et al. [9] examined cyclic arbitrage execution on L2s by analyzing on-chain bots, while Gogol et al. [4] and Öz et al. [15] explored non-atomic MEV across rollups. Öz et al. [6] highlights that on First Come, First Served (FCFS) L1s transaction-ordering techniques commonly exploited for MEV extraction on blockchains with fee-based prioritization do not directly translate to systems where ordering is determined solely by transaction arrival times.

Contributions. This paper advances the understanding of MEV in rollups with private mempools. Our main contributions are:

(i) Empirical. We conduct an empirical study of sandwich activity across major rollups with private mempools, using strict attacker-linking heuristics that distinguish EOAs, routers, and smart accounts [7]. To our knowledge, this is the first measurement of L2 sandwich activity that applies this methodology and *swap-event* analysis rather than ERC20 token-transfer inference.

Across all L2s, we find no evidence of sustained or economically meaningful sandwich attacks. Naive pattern-based heuristics vastly overstate activity: over 95% of flagged triples fail economic consistency checks, median net profit-and-loss (PnL) is negative, and sandwich efficiency remains below 0.05 per 100 transactions.

(ii) Theoretical. We extend the classical CPMM model to concentrated-liquidity AMMs. We show that the attacker optimal strategy depends on the tick boundaries and the liquidity distribution. In particular, attackers benefit most when pushing trades into thinner liquidity regions, which is in practice executed with flash-loans. We complement this framework with an execution-feasibility model that quantifies co-inclusion constraints under private mempools.

(iii) Execution feasibility under private mempools. We analyze the execution constraints imposed by centralized sequencers and private mempools. By formalizing co-inclusion probabilities as a function of block time and tip, we show that L2 sandwiching is inherently probabilistic rather than atomic, sharply limiting practical feasibility in today's rollup architectures.

Taken together, our findings inform rollup and sequencer design: the empirical absence of sandwiches on current L2s arises from private mempools and medium-value swaps rather than from inherent structural protections. As ecosystems transition toward public mempools, order-flow auctions, or higher-value trades, sandwich attacks may re-emerge, and sequencing policies should be evaluated with these economic risks explicitly in scope.

2 Sandwich Profitability and Execution Probability

Sandwich attacks exploit predictable price impact: an adversary buys before a victim's X→Y swap and sells after the victim moves the price upward. This section provides a concise analytical model for optimal attack sizing under both constant-product (CPMM) and concentrated-liquidity (CLMM) AMMs, followed by a quantitative execution-feasibility model for private-mempool rollups.

2.1 Economic Model

We model sandwich profitability across CPMMs and CLMMs in the small-trade regime characteristic of today's rollups, where both attacker and victim trades are typically small relative to tick-level liquidity. The objective is to derive the attacker's optimal frontrun size V_f as a function of the victim's input V_v, the AMM's liquidity profile, and the victim's slippage constraint. Throughout, we measure profits in USD-equivalent units.

Setting. An attacker executes a frontrun trade (X→Y), followed by the victim's X→Y trade, and completes with a backrun (Y→X). Let ϕ denote the proportional swap fee and L the effective liquidity depth of the AMM at the current price. In CPMMs, L corresponds to the reserve x_0; in CLMMs, L corresponds to the tick-level liquidity L_i/P. In both cases, the attacker's profit arises from the temporary wedge in exchange rate generated by the victim's price impact.

Small-trade regime. Define the normalized input sizes

$$\alpha_f = \frac{(1-\phi)V_f}{L}, \qquad \alpha_v = \frac{(1-\phi)V_v}{L}, \tag{1}$$

and assume $\alpha_f,\, \alpha_v \ll 1$, which empirically holds for most candidate patterns on rollups. Expanding the AMM swap function to second order in (α_f, α_v) yields the following result.

Proposition 1 (Small-trade approximation on CPMMs). *Consider a constant-product AMM with proportional fee ϕ and depth L. Let V_f and V_v denote the attacker and victim inputs. In the small-trade regime, the attacker's incremental profit (conditioning on the presence of the victim) admits the quadratic approximation*

$$\Delta\Pi(V_f; V_v) \;\approx\; \frac{(1-\phi)^2}{L}\big(V_f V_v - V_f^2\big) \;-\; 2\phi V_f, \tag{2}$$

where the term $2\phi V_f$ represents the swap fees paid on the frontrun and backrun legs. The unique interior maximizer is

$$V_f^\star = \frac{V_v}{2}. \tag{3}$$

The effective optimizer is $V_f^\star = \min\{\, V_v/2,\; V_f^{\max}\,\}$, where $V_f^{\max}$ is the upper bound implied by the victim's slippage tolerance.

Proposition 1 recovers the classical "half-the-victim" rule: the frontrun should be approximately half as large as the victim trade. As shown later, most L2 candidate sandwiches fail this proportionality condition, suggesting non-adversarial origins.

Extension to CLMMs. Concentrated-liquidity AMMs partition the price axis into ticks $[P_i, P_{i+1}]$, each endowed with constant liquidity L_i. Inside a single tick, the local depth is $L = L_i/P$, and the CPMM approximation applies verbatim. However, unlike CPMMs, CLMM liquidity is piecewise constant across ticks, producing discontinuities in marginal price impact at tick boundaries.

Proposition 2 (Optimal frontrunning across CLMM ticks). *Let the current price lie in tick $[P_i, P_{i+1}]$ with liquidity L_i. If the combined effective input of the frontrun and victim remains inside this tick, the optimal frontrun is the*

CPMM solution $V_f^\star = \min\{V_v/2,\, V_f^{\max}\}$. *If the combined path reaches the boundary and the next tick has lower liquidity* ($L_{i+1} < L_i$)*, the attacker strictly benefits from nudging the victim into the thinner region. In that case, the global optimum is*

$$V_f^\star = \min\{V_f^{\text{gap}},\, V_f^{\max}\}, \tag{4}$$

where V_f^{gap} *is the minimal frontrun size required to push the joint flow across the boundary. If* $L_{i+1} \geq L_i$*, no profit jump occurs and the within-tick optimizer remains optimal.*

Proposition 2 shows that CLMM sandwiches are *piecewise quadratic*: CPMM-like inside ticks, but with discrete profit jumps at boundaries with declining liquidity. This structure implies that genuine CLMM sandwiches should exhibit highly specific scaling patterns—either near $V_f \approx V_v/2$ (in-tick) or clustering around V_f^{gap} (boundary crossing). Deviations from both patterns, as shown empirically in Section 4, are indicative of false positives or non-sandwich MEV flows.

Empirical profit approximation. The small-trade CLMM profit before gas fees can be expressed compactly as

$$\Pi_{\text{net}} \;\approx\; (1-\phi)^2\,\frac{\varepsilon}{L}\left(V_f V_v - V_f^2\right) \;-\; 2V_f\phi, \tag{5}$$

where V_v denotes the victim's input, V_f the attacker's input per leg, ϕ the proportional swap fee (e.g. $\phi = 0.0005$ for 5 bps), L the real liquidity available in the active tick, and $\varepsilon \approx 1.0001^{\Delta\text{tick}} - 1$ the relative width of the tick. For the common case of $\Delta\text{tick} = 1$ and small ϕ, this reduces to

$$\Pi_{\text{net}} \;\approx\; \frac{V_f(V_v - V_f)}{L}\cdot 10^{-4} \;-\; 2V_f\phi,$$

which we use as the empirical profit approximation before gas fees to assess the (un)profitability of candidate sandwiches in observed data. Further, we report profits both before after gas fees.

2.2 Execution Probability Under Private Mempools

Even if the economic optimum $V_f^\star$ is well defined, executing a sandwich requires that the attacker's frontrun (f), victim transaction (v), and backrun (b) all appear in the same block (or adjacent micro-batches) and in the correct order. On rollups with private mempools, no builder market guarantees atomic inclusion of (f, v, b). Instead, execution becomes *probabilistic* and depends critically on the sequencer's ordering rule. Most L2s implement two canonical policies:

- *(1) First Come, First Served (FCFS; time-priority):* transactions are ordered primarily by arrival time, with tips used only as tie-breakers in micro-batches,

– *(2) Priority Gas Acution (PGA; tip-priority):* transactions within a batch are ordered by descending priority fee. Arrival time affects only which batch a transaction enters.

This distinction is central: under FCFS, attackers optimize *timing* (Δ), while under PGA they optimize *tips* (t_f, t_b) and face a Bayesian "tip-guessing" problem relative to the victim and background flow. We now formalize the components of execution probability in a framework that accommodates both rules. Let

– T_b = block time,
– T_s = sequencer batching window,
– Δ = time between f and b submissions (attacker choice under FCFS),
– t_f, t_v, t_b = priority tips on front-, victim-, and back-run,
– $F(\cdot)$ = sequencer ordering function ($F(t)$ increasing in tips under PGA),
– σ = network/latency variance.

Same-batch probability. For both FCFS and PGA, f and b must fall into the same batch for a single-block sandwich. A simple approximation is

$$p(\text{batch}) \;\approx\; \max\left(0,\, 1 - \frac{\Delta}{T_s}\right). \tag{6}$$

Under FCFS, Δ is a strategic choice; under PGA, attackers typically send f and b simultaneously, so $\Delta \approx 0$.

Priority-ordering probability: FCFS vs. PGA. A valid sandwich requires $f \prec v \prec b$ under the sequencer's internal ordering rule. Even conditional on $t_f > t_v > t_b$, the ordering constraint $f \prec v \prec b$ may fail if background transactions with intermediate priority fees are inserted between the attacker's legs.

FCFS. Ordering is determined (up to noise) by arrival times. Let N be the number of intervening transactions between submission times of f and b. If background transactions arrive according to a Poisson process with rate λ, then

$$p_{\text{FCFS}}(\text{priority}) = \Pr[N = 0] = e^{-\lambda\Delta}.$$

Here the attacker optimizes Δ: shorter delays reduce the probability of interference.

PGA. Ordering is determined by priority tips. A correct sandwich requires $t_f > t_v > t_b$. Let background tips be i.i.d. from distribution G. If K transactions enter the batch with v, then the probability that none fall in the dangerous interval (t_b, t_f) is

$$p_{\text{PGA}}(\text{priority}) = \mathbb{E}_{t_v,K}\big[\mathbf{1}\{t_f > t_v > t_b\}(1 - (G(t_f) - G(t_b)))^K\big].$$

Under PGA, timing does not improve relative ordering; only tip selection matters, and the attacker faces a Bayesian optimisation over (t_f, t_b) given uncertainty about t_v.

Arrival-time probability. Arrival times are noisy. If the effective delay between intended and realized arrival times is Gaussian with variance σ^2, then

$$p(\text{arrival}) \;\approx\; \exp\!\left(-\frac{\Delta^2}{2\sigma^2}\right). \tag{7}$$

This component is relevant primarily under FCFS, where Δ is a choice variable; under PGA, Δ is typically negligible.

Co-inclusion probability. Combining the components yields the attacker's feasibility frontier:

$$p(\text{co-inc}) \;\approx\; \left(1-\frac{\Delta}{T_s}\right) p_{\text{policy}}(\text{priority}) \exp\!\left(-\frac{\Delta^2}{2\sigma^2}\right), \tag{8}$$

where p_{policy} is p_{FCFS} or p_{PGA} depending on the sequencer rule.

For typical rollup parameters ($T_b \approx 200$–$500\,\text{ms}$, $T_s \approx 300$–$800\,\text{ms}$, $\sigma \approx 50\,\text{ms}$), the resulting co-inclusion probabilities lie between 0.05 and 0.20 across both FCFS and PGA systems, well below the near-atomic inclusion achievable on Ethereum L1 via bundle markets. Consequently, even economically optimal sandwiches have low expected value on today's private-mempool L2 architectures.

2.3 Minimum Victim Swap Size for Positive Expected Value on L2s

The attacker's expected profit on a rollup with private mempools is the product of (i) the economic profit generated by the AMM price impact and (ii) the probability of co-inclusion under the sequencer's ordering rule. From Section 2.1, under the small-trade approximation the attacker's optimal frontrun size satisfies $V_f^\star \approx V_v/2$, yielding gross profit

$$\Pi^\star_{\text{gross}}(V_v) \;\approx\; \frac{1}{4L} V_v^2, \tag{9}$$

where L is the effective liquidity depth of the active tick in the CLMM. However, as shown in Section 2.2, in private-mempool rollups the sandwich executes only with probability $p_{\text{succ}} \in [0.05, 0.20]$, depending on batching windows, tip ordering, and arrival-time variance. The attacker's expected value (EV) satisfies

$$\mathbb{E}[\Pi] = p_{\text{succ}} \cdot \Pi^\star_{\text{gross}}(V_v) - C_{\text{gas}} - C_{\text{slip}}, \tag{10}$$

where C_{gas} denotes L2 gas fees for the front- and back-run (typically \$0.20–\$0.70 in total) and C_{slip} is the slippage loss incurred by the attacker's own trades.

A sandwich becomes profitable only when $\mathbb{E}[\Pi] > 0$, which implies the following minimum victim trade size:

$$V_v^{\min} \;=\; 2\sqrt{L \cdot \frac{C_{\text{gas}} + C_{\text{slip}}}{p_{\text{succ}}}}. \tag{11}$$

Under co-inclusion probabilities $p_{\text{succ}} \in [0.05, 0.20]$ and typical active tick liquidity L between \$50k and \$300k, Eq. (11) implies that victim swap sizes must generally exceed \$1.5k–\$3k. These thresholds exceed by a factor of $2\times$–$10\times$ the median victim swap sizes observed in Section 5 for major rollups (typically \$200–\$1,200). Consequently, even economically optimal sandwiches would yield negative expected value on today's L2s.

2.4 Execution Feasibility

Executing a sandwich attack requires that a transaction f front-runs a set of victim transactions v_i both of which are followed by a back-running transaction b. Here, we outline methods by which someone can attempt a sandwich attack.

While it is relatively easy to establish that f should be the first transaction of a block (e.g., by paying the highest priority fee), it is more difficult to ensure that b comes after some victim transactions. Necessarily, the attacker must send f and b in two different transactions to the target blockchain and the nonce of b must be higher than the nonce of f if b is to be considered valid. This is also sufficient though it does not guarantee that both f and b appear in the same block. Assuming that block builders sort by priority fee, it is necessary to have that the priority fee of b is not larger than the priority fee of f (and larger fee gaps provide more opportunities for victim transactions between b and f). In order to execute b after f, two straightforward approaches could be used:

- Introducing an artificial delay before sending b to the blockchain. This may work under FCFS-style sequencing on blockchains with relatively slow block production times (e.g., block times greater than 1s), but is generally ineffective under PGA-style ordering, where delaying b weakens batch co-inclusion without improving relative ordering. On fast-rollup chains (e.g., Base with $\sim$ 200ms block times), such delays often prevent same-block execution altogether.
- Sequentially submitted calls without a delay but different tip values via an intermediary contract C which contains the necessary logic for both front-running and back-running. A function may be placed behind a modifier which chooses which logic to execute based on the contract state and current block.

3 Methodology

Data Collection. We collected all swap event logs with corresponding transactions and transaction traces from Arbitrum, Base, Optimism, Unichain and ZKsync from January to September 2025, using Dune queries. For profit analysis, we extract pool-level liquidity (for CPMMs), current and surrounding ticks liquidity (for CLMMs), and the applicable LP fee tier for each pool.

Sandwich Pattern Heuristics. A central challenge in detecting sandwich attacks is reliably linking the front- and back-running legs to the same adversarial actor. A naïve method is to use the `taker` field from pool-level `Swap` events,

which records the immediate caller of the pool contract. While convenient, this has two major drawbacks: (i) it often resolves to shared routers or aggregators (e.g., UniswapRouter, 1inch) that serve many independent users, and (ii) it may differ across the front- and back-leg if the attacker routes through multiple entrypoints. As a result, `taker`-based attribution misses valid sandwiches and produces false positives. For instance, when shared routers collapse unrelated users into a single calling address, two benign trades may appear as a coordinated front–back pair. To overcome this, we derive an adversary identifier $\mathsf{actor_id}$ (as described in Step 3 below) and detect candidate sandwich patterns using the following procedure:

1. **Pre-filtering of atomic arbitrage.** Exclude single transactions with cyclic (atomic) arbitrage transactions, as they may mimic sandwich legs. We follow the methodology of [12] for detection.
2. **Partitioning.** Group all swaps by (`blockchain`, `block_number`, $\mathsf{pool_address}$) so that only transactions interacting with the same liquidity pool in the same block are compared. For Uniswap v4, we extend the grouping with the currency pair, since event logs register only the router address.
3. **Actor attribution.** Assign each transaction an $\mathsf{actor_id}$ according to the following rule. If the top-level callee `tx_to` is a known system contract (e.g., ZKSync bootloader) or a widely used router/aggregator, we set $\mathsf{actor_id} :=$ `tx_from`, which refers to the funding externally owned account (EOA) or smart account. This captures adversaries who execute via shared infrastructure. Otherwise, if the callee `tx_to` is a custom contract, we set $\mathsf{actor_id} :=$ `tx_to`. This captures adversaries using dedicated executor contracts.
4. **Ordering constraint.** Within each group, order transactions by block index and select triples (f, v_i, b) that satisfy $\text{index}(f) < \text{index}(v_i) < \text{index}(b)$ and $\mathsf{actor_id}(f) = \mathsf{actor_id}(b) \neq \mathsf{actor_id}(v_i)$.
5. **Swap-direction constraint.** Require that the trade direction of the victim and front legs match, while the back leg has the reverse direction.

4 Empirical Evidence

Before presenting the empirical evidence, we briefly connect the theoretical results of Section 2 to the measurement strategy. The quadratic profit model predicts that genuine sandwiches exhibit two key signatures: (i) attacker legs scale proportionally with the victim ($V_f^\star \approx V_v/2$ inside a tick), and (ii) front- and back-run sizes should be closely matched except at tick boundaries with declining liquidity. At the same time, the execution-feasibility model implies that on private-mempool rollups only a small fraction of attempted sandwiches can achieve correct ordering, even when economically optimal.

These theoretical constraints motivate the economic-consistency checks used in this section and explain why most observed "sandwich-shaped" triples are expected to be false positives or arbitrage by-products rather than profit-maximizing attacks.

Table 1. Summary statistics of swap sizes (USD) for front, victim, and back legs. For each role we report the median and the interquartile range (IQR).

		Median			IQR		
Chain	Count	Front	Victim	Back	Front	Victim	Back
Arbitrum	2,576	807.1	872.8	1077.1	[400.3, 1960.7]	[403.2, 2391.9]	[500.4, 2662.7]
Base	50,952	2230.5	1224.0	2157.7	[911.9, 4060.5]	[398.8, 2999.0]	[905.8, 3825.0]
Optimism	177	464.1	1103.0	309.2	[211.4, 1469.8]	[447.9, 3167.1]	[179.5, 640.5]
Unichain	795	682.5	3045.4	546.8	[273.1, 3662.3]	[500.7, 6449.8]	[222.9, 2934.0]
ZKsync	27	441.4	400.6	194.2	[255.8, 665.3]	[249.4, 1729.1]	[153.2, 430.0]

4.1 Swap Size Analysis

Table 1 reports descriptive statistics of swap sizes for candidate sandwich triples across studied rollups. Base dominates the sample with over 50,000 identified triples, while Optimism and ZKsync have limited coverage.

Across chains, the relative sizing of victim and attacker legs is inconsistent. On Base, for instance, the median victim size is $1.2k, while the corresponding front- and back-runs are larger, around $2.2k and $2.1k respectively. On Optimism and ZKsync, front- and back-runs are frequently much smaller than victims, while on Unichain the opposite pattern appears, with victims typically several times larger than attacker legs. Importantly, across all chains, these swaps are small in absolute terms, with median values on the order of only a few hundred to a few thousand USD, far below the scale typically associated with profitable sandwiches on Ethereum L1 [7,11]. This lack of consistent scaling between attacker and victim swap sizes, combined with their generally modest magnitudes, foreshadows our later finding that most of these triples are not genuine sandwiches but are better explained by arbitrage activity or unrelated transactions coinciding in the same block.

False Positives. Table 2 presents cross-chain statistics on candidate sandwich triples. "Strong Sig." denotes cases, in which backrun swap size does not deviate by more than 10% from the frontrun swap. Reported correlations are Pearson coefficients. The relatively low share of Strong Signature cases (5–25%) and consistently weak correlations among front-, back-running and victim swaps indicate that most candidate triples are not genuine victim-targeted sandwiches.

Back vs. Front Size Mismatch. Median back/front ratios deviate substantially from 1.0, with wide dispersion across chains. For example, the median ratio is 0.728 on Optimism (IQR $[0.282, 1.526]$) and 0.825 on Unichain (IQR $[0.358, 1.913]$). True sandwiches should cluster near 1.0 (back $\approx$ front). Instead, attackers leave large mismatches, consistent with external hedging such as a CEX leg. Figure 1 highlights these deviations, showing distributions dispersed and biased away from zero.

Front vs. Back Correlation is Weak. Front$\leftrightarrow$back correlations remain far below 1.0. For instance, the correlation is only 0.08 on Optimism and 0.40

Table 2. Cross-chain summary of candidate sandwich triples. "Strong Sig." denotes cases of back/front (b/f) size matching criteria. Reported correlations are Pearson coefficients. The relatively low shares of Strong Signature cases and weak correlations across chains suggest that most candidate triples are not genuine victim-targeted sandwiches.

Chain	Strong Sig.	Median b/f	IQR b/f	Victim ↔Front	Victim ↔Back	Front ↔Back
Arbitrum	24.6%	1.035	[0.816, 2.672]	0.426	0.409	0.627
Base	14.3%	0.967	[0.549, 1.583]	0.276	0.375	0.352
Optimism	5.1%	0.728	[0.282, 1.526]	0.032	0.592	0.082
Unichain	7.3%	0.825	[0.358, 1.913]	0.162	0.182	0.398
ZKsync	7.4%	0.674	[0.392, 0.785]	0.081	0.567	0.354

on Unichain, with the highest case 0.63 on Arbitrum. True sandwiches require matched trade sizes, but the observed weak dependence suggests that front and back trades are not coordinated pairs, rather independent legs of broader arbitrage strategies.

Victim Trade has Little Influence on Attacker Size. Correlations between victim trades and attacker trades are consistently weak. Victim↔front correlations are as low as 0.03 on Optimism and only 0.16 on Unichain; victim↔back correlations reach 0.18 on Unichain and 0.57 on ZKsync but remain far from 1. In genuine sandwiches, attacker trades should scale with victim size. The absence of such scaling indicates that attacker activity is not driven by exploiting victim slippage.

4.2 Negative Net Profits

We report three profitability measures for each candidate sandwich attack. The first, *median_pnl_gross_usd*, represents the potential gain from exploiting the victim's price impact before accounting for the attacker's own costs. The second, *median_pnl_net_slippage_usd*, adjusts this value by incorporating the attacker's own slippage losses, capturing the fact that executing front- and backrun trades also moves the price against them. The third, *median_pnl_net_usd*, further deducts transaction fees and observed gas expenditures, providing the most conservative estimate of realized profitability.

Empirically, as summarized in Table 3, median gross profits are on the order of 10^{-3} USD per attack and are almost entirely eliminated once slippage is considered, with final median net PnL turning negative across all rollups. Figure 2 further illustrates that these values are tightly clustered around zero with heavy tails, reinforcing that candidate sandwiches do not generate sustainable profits in practice and that the vast majority of observed patterns are better explained by arbitrage-like or other MEV activity rather than deliberate victim targeting.

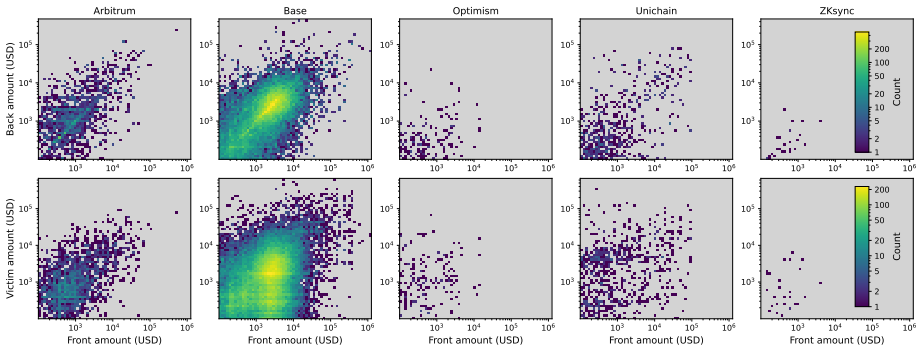


Fig. 1. Density heatmaps of swap sizes (USD) for front, victim, and back legs. The top row plots back-run versus front-run swaps; a genuine sandwich would concentrate mass along the diagonal $b = f$, yet observed densities scatter widely away from it. The bottom row plots victim versus front-run swaps; here too, the absence of clustering along the diagonal $v = f$ indicates that attacker trades do not scale with victim size.

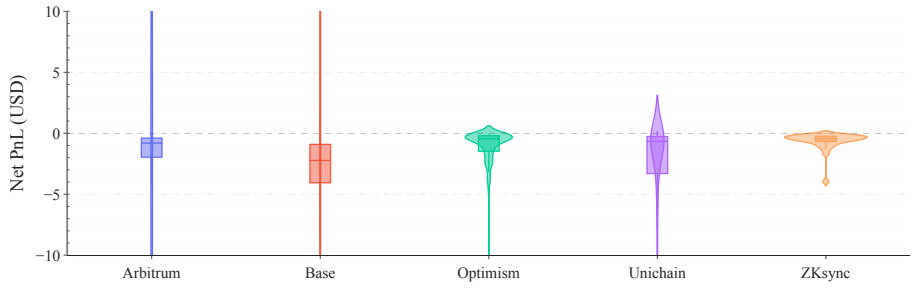


Fig. 2. Violin distribution of estimated net PnL in USD. The distributions are tightly centered around zero, with heavy tails, confirming that median profits are negligible and most candidate sandwiches are not profitable.

Table 3. Summary of estimated Profit-and-Loss (PnL) in USD per candidate sandwich across rollups. Gross PnL is the attacker's profit before costs, Net Slippage PnL accounts for execution price impact only, and Net PnL further deducts gas fees. Medians across all L2s are close to zero, and interquartile ranges (IQR) for Net PnL confirm that most sandwich patterns cluster around small losses.

Chain	Median Gross PnL	Median Net Slippage	Median Net PnL	IQR Net PnL
Arbitrum	0.0005	0.0000	−0.8026	[−1.9629–−0.4002]
Base	0.0018	−0.0005	−2.2278	[−4.0593–−0.9106]
Optimism	0.0010	0.0001	−0.4569	[−1.4666–−0.2069]
Unichain	0.0027	0.0004	−0.6521	[−3.2985–−0.2707]
ZKsync	0.0009	0.0000	−0.4415	[−0.6587–−0.2558]

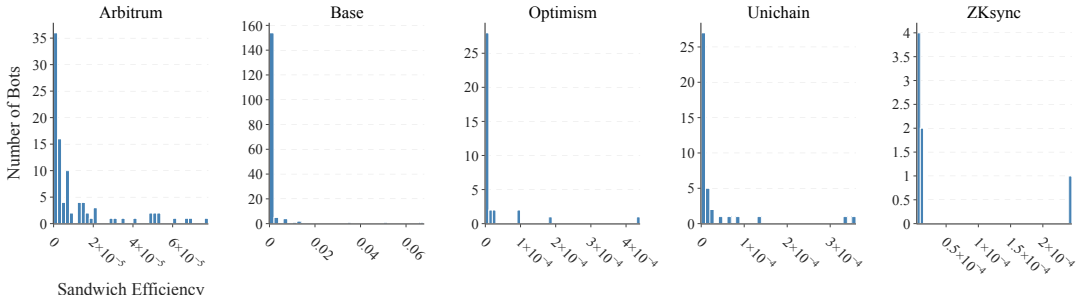


Fig. 3. Histograms of sandwich efficiency per bot across L2s (Jan–Sep'25). Efficiency is defined as the share of sandwiches among a bot's total transactions. Across all rollups, the vast majority of bots exhibit extremely low efficiency, indicating that most sandwich-like patterns are not genuine MEV attacks.

Table 4. Per-chain activity of bot addresses flagged as sandwich initiators. The large number of daily transactions compared to the relatively low cumulative number of sandwiches per bot suggests that most cases are accidental by-products of broader bot activity. Sandwich counts and averages are reported per bot.

Chain	# Bots	Sandwiches (Jan–Sep'25)			Avg. Daily Tx		
		Min	Median	Max	Min	Median	Max
Arbitrum	96	1	5	321	55	5,968	382,739
Base	169	1	14	11,614	72	6,315	2,465,363
Optimism	36	1	3	37	19	5,324	223,562
Unichain	40	1	6	175	649	3,778	248,500
ZKsync	7	1	1	15	334	609	1,434

4.3 Bot Activity Characteristics

Table 4 summarizes the characteristics of addresses flagged as sandwich attackers across the studied L2s. For each rollup we report the number of distinct attackers, the total number of sandwich attempts observed during the Jan–Sep'25 window, and the per-attacker distributions of sandwiches and daily transactions. The data reveal stark heterogeneity: while ZKsync shows only 7 addresses with a combined 27 sandwiches, Arbitrum and Base each host dozens to hundreds of bots with thousands of attempts. Transaction activity is equally skewed, with daily averages ranging from a few dozen to several hundred thousand per attacker. Outliers such as a single Arbitrum bot with 382,739 transactions per day or a Base bot with 11,614 sandwiches dominate aggregate counts, indicating that sandwiching is highly concentrated.

To further examine how specialized these bots are, we measure *sandwich efficiency*, defined as the share of sandwiches among a bot's total transactions. Figure 3 shows the histogram of bots' efficiency. In all chains, the median efficiency remains extremely low—often below 10^{-3}—with only a few long-tailed

cases of concentrated sandwiching. This suggests that the majority of addresses identified by heuristics are not dedicated sandwichers, but instead engage in broader arbitrage or market-making strategies, with sandwiches representing a marginal fraction of their flow. The joint evidence from Table 4 and Figure 3 underscores that sandwich attacks on L2s are rare, sporadic, and typically overshadowed by other forms of trading activity.

5 Discussion and Conclusions

Sandwich attacks on Ethereum L1 have been extensively studied and are known to be both common and profitable. Their success relies on three structural features: deep liquidity that supports large victim trades, open builder markets that guarantee atomic inclusion of attacker legs, and transparent mempools that allow precise ordering control. These conditions collectively support a robust ecosystem of searchers who consistently extract value through sandwiches.

Our measurements show that none of these enabling conditions hold on L2 rollups. Victim trades are an order of magnitude smaller, attacker legs fail to scale with victim size, and nearly all candidate patterns are economically inconsistent once slippage and fees are accounted for. Even when an attacker guesses the victim's pool and routing path correctly, centralized sequencing and private mempools eliminate atomic guarantees and reduce execution to a probabilistic event. The low co-inclusion probability, combined with shallow liquidity and narrow tick ranges, sharply limits feasible returns.

These findings have two key implications. First, heuristic detection methods may substantially overestimate the prevalence of sandwiches on rollups by conflating innocent arbitrage with genuine victim-targeted attacks. Careful interpretation is required when labeling sandwich-like patterns in private mempool environments. Second, L2 defense mechanisms should not inherit L1-style assumptions: mitigation strategies must distinguish victim-targeted ordering manipulation from benign arbitrage flow, and policy discussions should reflect the fundamentally different sequencing model used by rollups.

Limitations. Our analysis focuses on same-block patterns; on rollups with subsecond block times, cross-block sandwiches are theoretically possible, though empirically rare. Actor attribution relies on transaction-tree heuristics and may merge unrelated flows, particularly when aggregator contracts bundle multiple hops. Finally, our dataset covers the major rollups but may miss smaller venues or pools with different characteristics.

Conclusions. This paper provides the first systematic analysis of sandwich attacks under private L2 mempools. We find that while the theoretical structure of sandwich profitability extends to rollups, practical execution does not: liquidity depth, slippage bounds, and low co-inclusion probability sharply limit feasible gains. Empirically, approximately 95% of sandwich-shaped triples fail basic economic consistency checks, median profitability is negative, and attacker addresses exhibit negligible sandwich efficiency.

These results challenge the common assumption that sandwich behavior on Ethereum L1 generalizes to L2 environments. In private mempools controlled by centralized sequencers, sandwiches are rare, unprofitable, and often indistinguishable from neutral arbitrage. As L2 sequencing markets evolve—particularly if decentralized builders or preconfirmation markets emerge—the feasibility frontier for sandwiching may shift. Future work should monitor these changes, expand measurements to additional rollups, and explore cross-block patterns or searcher strategies that lie outside the classical same-block sandwich model.

References

1. Alipanahloo, Z., Hafid, A.S., Zhang, K.: Maximum extractable value (MEV) mitigation approaches in Ethereum and layer-2 chains: A comprehensive survey. IEEE Access **12**, 185212–185231 (2024)
2. Chaliasos, S., Firsov, D., Livshits, B.: Towards a formal foundation for blockchain rollups (2024), https://arxiv.org/abs/2406.16219
3. Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., Juels, A.: Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In: SP. pp. 910–927. IEEE (2020)
4. Gogol, K., Messias, J., Miori, D., Tessone, C., Livshits, B.: Cross-rollup MEV: Non-atomic arbitrage across L2 blockchains (2024), https://arxiv.org/abs/2406.02172
5. Heimbach, L., Wattenhofer, R.: SoK: Preventing transaction reordering manipulations in decentralized finance. In: AFT. pp. 47–60. ACM (2022)
6. Öz, B., Rezabek, F., Gebele, J., Hoops, F., Matthes, F.: A study of MEV extraction techniques on a first-come-first-served blockchain. In: SAC. pp. 288–297. ACM (2024)
7. Qin, K., Zhou, L., Gervais, A.: Quantifying blockchain extractable value: How dark is the forest? pp. 198–214 (2022)
8. Sguanci, C., Spatafora, R., Vergani, A.M.: Layer 2 blockchain scaling: a survey (2021), https://arxiv.org/abs/2107.10881
9. Solmaz, O., Heimbach, L., Vonlanthen, Y., Wattenhofer, R.: Optimistic MEV in Ethereum layer 2s: Why blockspace is always in demand (2025), https://arxiv.org/abs/2506.14768
10. Thibault, L.T., Sarry, T., Hafid, A.S.: Blockchain scaling using rollups: A comprehensive survey. IEEE Access **10**, 93039–93054 (2022)
11. Torres, C.F., Camino, R., State, R.: Frontrunner Jones and the Raiders of the Dark Forest: An Empirical Study of Frontrunning on the Ethereum Blockchain. In: 30th USENIX Security Symposium (USENIX Security 21) (2021)
12. Torres, C.F., Mamuti, A., Weintraub, B., Nita-Rotaru, C., Shinde, S.: Rolling in the shadows: Analyzing the extraction of MEV across layer-2 rollups (2025), https://arxiv.org/abs/2405.00138
13. Wang, H., Zhang, J., Birge, J.R.: Profitability of collusive sandwich attack in automated market maker-based decentralized exchanges (2023). https://doi.org/10.2139/ssrn.5274901
14. Zhou, L., Qin, K., Torres, C.F., Le, D.V., Gervais, A.: High-frequency trading on decentralized on-chain exchanges (2020), https://arxiv.org/abs/2009.14021
15. Öz, B., Torres, C.F., Schlegel, C., Mazorra, B., Gebele, J., Rezabek, F., Matthes, F.: Cross-chain arbitrage: The next frontier of MEV in decentralized finance (2025), https://arxiv.org/abs/2501.17335